

HW6: Some results of the spot reservation

1. Check 1: Testing Permit Issuance

```
43
44
45 -- 1: Issuing a Permit using the "issue_permit_secure" function
46 -- Testing a successful issuance
47 SELECT issue_permit_secure('ABC-123', '2026-12-31', 'Commuter');
48 -- Verify it was added
49 SELECT * FROM permits WHERE license_plate = 'ABC-123';
50
51
52
53
```

Data Output Messages Notifications

Showing rows: 1 to 3

	permit_id [PK] integer	license_plate character varying (20)	expiry_date date	permit_type character varying (50)
1	1	ABC-123	2026-12-31	Commuter
2	9	ABC-123	2025-01-01	EXPIRED_TEST
3	12	ABC-123	2026-12-31	Commuter

2. Check 2: Testing to see if a parking spot changes from available to occupied when user parks vehicle. (NOTE: I'm testing with spot_id #5, look for a spot_id that is "Available" unless it won't work as expected)

```
53
54
55 -- 2: Changing a spot to occupied when a user parks in there
56 -- Check the initial status of the spot
57 SELECT spot_id, current_status FROM spots WHERE spot_id = 5;
58 -- Insert sensor event (this is what triggers the update)
59 INSERT INTO sensor_events (spot_id, entry_time) VALUES (5, NOW());
60 -- Check updated status
61 SELECT spot_id, current_status FROM spots WHERE spot_id = 5;
62
63
64
65
66
67
68
```

Data Output Messages Notifications

	spot_id [PK] integer	current_status character varying (20)
1	5	Available

- Before

- After

```

54
55 -- 2: Changing a spot to occupied when a user parks in there
56 -- Check the initial status of the spot
57 SELECT spot_id, current_status FROM spots WHERE spot_id = 5;
58 -- Insert sensor event (this is what triggers the update)
59 INSERT INTO sensor_events (spot_id, entry_time) VALUES (5, NOW());
60 -- Check updated status
61 SELECT spot_id, current_status FROM spots WHERE spot_id = 5;
62
63
64
65
66
67
68

```

Data Output Messages Notifications

Showing rows: 1 to 1

	spot_id [PK] integer	current_status character varying (20)
1	5	Occupied

3. Check 3: Testing for violations and sending tickets

- We need reservation to be expired but spot still occupied. Since we already occupied spot 5 in the above example, we won't need to reoccupy it, but we will use that parking spot.
- NOTE: I've updated some information for my generate_tickets() function

```

65 -- 3: Testing for reservation time frame violation and updating
66 -- Since we just occupied spot_id 5, we can now set a reservation for it
67 UPDATE reservations
68 SET end_time = CURRENT_TIMESTAMP - INTERVAL '1 hour'
69 WHERE spot_id = 5;
70 -- Then we check the DB for a violation
71 CALL generate_tickets();
72 -- Then we make sure a ticket was actually generated for the correct parking spot violation
73 SELECT t.ticket_id, t.license_plate, t.violation_reason, t.amount, t.is_paid
74 FROM tickets t
75 JOIN vehicles v ON t.license_plate = v.license_plate
76 JOIN reservations r ON v.owner_id = r.user_id
77 WHERE r.spot_id = 5;
78
79
80
81
82
83
84
85

```

Data Output Messages Notifications

Showing rows: 1 to 2

	ticket_id [PK] integer	license_plate character varying (20)	violation_reason character varying (100)	amount numeric (10,2)	is_paid boolean
1	5	GHI-789	Blocking Hydrant	100.00	false
2	16	GHI-789	Overtime Parking	50.00	false

```

78 SELECT * FROM tickets WHERE ticket_id=5;
79
80
81
82
83
84
85

```

Data Output					
	ticket_id [PK] integer	license_plate character varying (20)	violation_reason character varying (100)	amount numeric (10,2)	is_paid boolean
1	5	GHI-789	Blocking Hydrant	100.00	false

- More clear ticket

4. Check 4: Concurrency Control. Multiple transactions at the same time

- Using Locks, we'll show how it's impossible for 2 people to book the same parking spot at the same time
- Session A:

```

80
81
82 -- 4: Concurrency Control
83 -- 4: Session A
84 -- Starting the transaction
85 BEGIN;
86 -- Lock the spot so no one else can touch it
87 SELECT * FROM spots WHERE spot_id = 6 FOR UPDATE; -- This area is now locked because this user is using it
88
89
90

```

Data Output				
	spot_id [PK] integer	lot_id integer	spot_number character varying (10)	current_status character varying (20)
1	6	6	F1	Available

Showing rows: 1 to 1

- Session B:

Query	Query History	AI Assistant
1		-- 4: Concurrency Control
2		-- 4: Session B
3		-- Also starting the transaction
4		BEGIN;
5		-- Trying to lock the same spot
6		SELECT * FROM spots WHERE spot_id = 6 FOR UPDATE; -- this should not work for the second user
7		
8		

Data Output	
Messages	
ERROR: current transaction is aborted, commands ignored until end of transaction block	
SQL state: 25P02	

```
82 -- 4: Concurrency Control
83 -- 4: Session A
84 -- Starting the transaction
85 BEGIN;
86 -- Lock the spot so no one else can touch it
87 SELECT * FROM spots WHERE spot_id = 6 FOR UPDATE; -- This area is now locked because this user is using it
88
89 COMMIT;
90
91
```

Data Output Messages Notifications

COMMIT

Query returned successfully in 103 msec.